COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	8/1/26	**DATE SUBMITTED:**	8/1/26
NAME:	Ralf Nicolo B. Abello	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1	**Stacks**

1. PROCEDURE OUTPUT

ILO A

Main.cpp

```cpp
#include <iostream>
#include <stack> // call the stack from the STL

int main(){
    // created an object
    std::stack<int> newStack;
    std::stack<int> stack1;
    std::stack<int> stack2;

    //testing the empty()
    std::cout<<"Is the stack empty? " <<newStack.empty()<<std::endl;

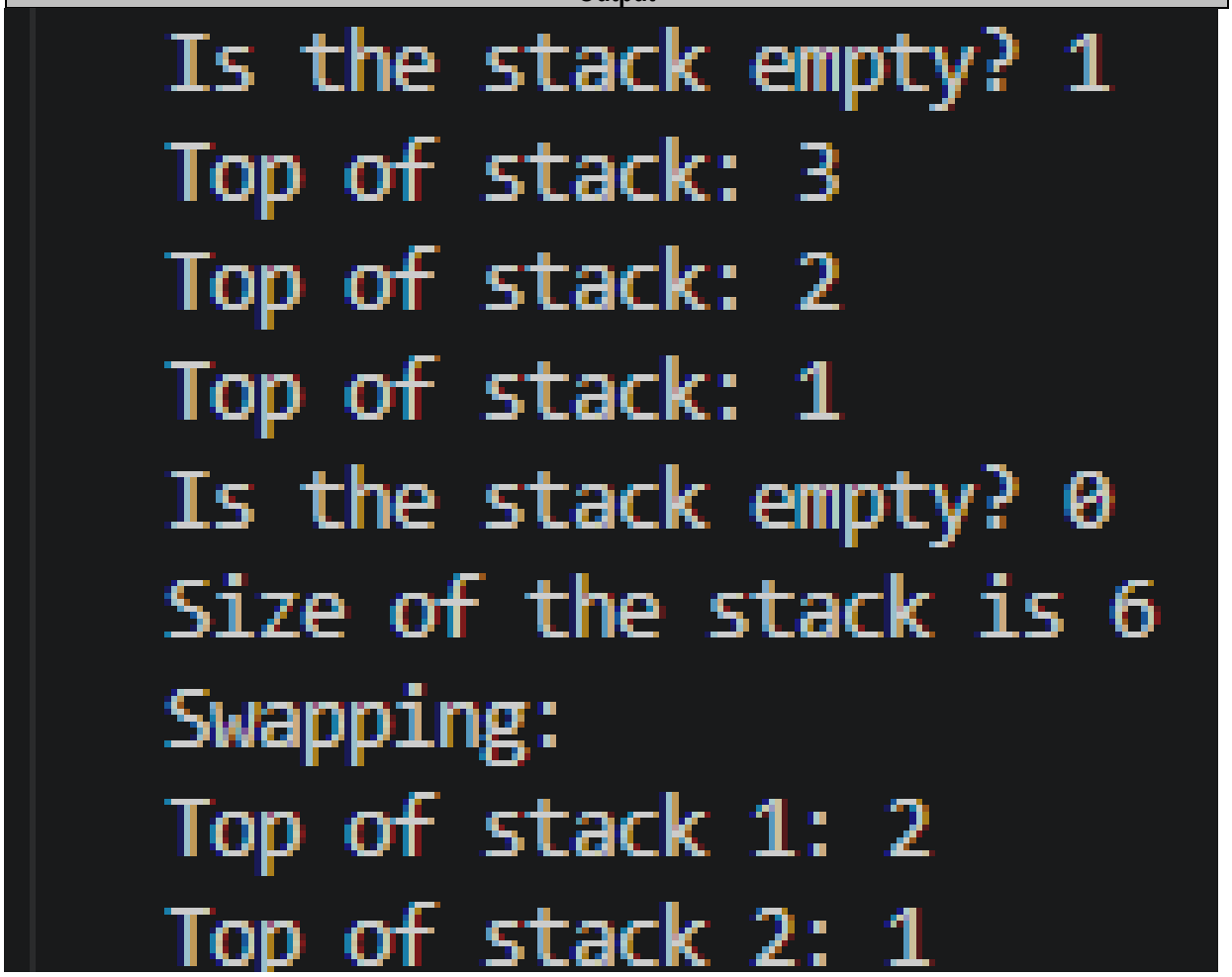
    //insert a 3 elements:
    newStack.push(3);
    std::cout<<"Top of stack: " <<newStack.top()<<std::endl;
    newStack.push(2);
    std::cout<<"Top of stack: " <<newStack.top()<<std::endl;
    newStack.push(1);
    std::cout<<"Top of stack: " <<newStack.top()<<std::endl;

    std::cout<<"Is the stack empty? " <<newStack.empty()<<std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);
    std::cout<<"Size of the stack is " <<newStack.size()<<std::endl;

    //swap
    stack1.push(1);
    stack2.push(2);
    stack1.swap(stack2);
    std::cout<<"Swapping: "<<std::endl;
    std::cout<<"Top of stack 1: " <<stack1.top()<<std::endl;
    std::cout<<"Top of stack 2: " <<stack2.top()<<std::endl;

}
```

Output



Code Analysis

For the ILO A I built this program to show how **std::stack** works. It's a container adapter, meaning it's built on top of another container, and by default that container is **std::deque**. I use it to show LIFO (last-in, first-out) order, where the last item I add is the first one I can see or remove. I start **newStack** empty, and calling **empty()** correctly prints **1**, since C++ prints true as **1** and false as **0** by default. Then I call **push(3)**, **push(2)**, and **push(1)** one at a time, and each call makes that new value the new top, so my three prints in a row correctly show **3**, then **2**, then **1**. After that, I push three more values with **push(6)**, **push(7)**, and **push(8)**, and calling **size()** correctly reports **6**, since I've pushed six values total and never removed any. I also test **swap**, where I give **stack1** one element using **push(1)** and give **stack2** one element using **push(2)**, then I call **stack1.swap(stack2)** to trade their entire contents at once, in constant time. After that swap, **stack1.top()** correctly prints **2** and **stack2.top()** correctly prints **1**, confirming the swap worked as expected. Overall, my logic is correct, my program compiles and runs exactly as intended, and it clearly demonstrates the core operations of **push**, **top**, **empty**, **size**, and **swap**, all in a single, straightforward, easy-to-follow example.

ILO B1

Main.cpp

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

//prototyping
bool isEmpty();
void push();
void pop();
void Top();
bool isFull();
void displayAll();

int main(){
    int choice;
    //asked the user for the maximum elements to be inserted
    std::cout<<"Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin>>elements;

    while(true){
        std::cout<<"Stack Operations: "<<std::endl;
        std::cout<<"1. PUSH \n2. POP \n3. TOP \n4. isEmpty \n5. isFull \n6. Display All"<<std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<<isEmpty()<<std::endl;
            break;
            case 5: std::cout<<isFull()<<std::endl;
            break;
            default: std::cout<<"Invalid choice"<<std::endl;
            case 6: displayAll();
            break;
        }
    }

    return 0;
}
```

```
45
46  bool isEmpty(){
47      //top is -1
48      if(top == -1){
49          std::cout<<"Stack Underflow"<<std::endl;
50          return true;
51      }
52      return false;
53  }
54
55  bool isFull(){
56      //not equal to the maxCap
57      if(top >= elements-1){
58          std::cout<<"Stack Overflow";
59          return true;
60      }
61      return false;
62  }
63
64  void push(){
65      //check if the stack is full, if yes return an error
66      if (isFull()) return;
67
68      //ask the user for a value to be pushed into the stack
69      std::cout<<"What number you want to add to the stack?: ";
70      std::cin >> newData;
71
72      //store inside the stack
73      stack[++top] = newData;
74  }
75
76  void pop(){
77      //check if there are no more elements inside the stack
78      if(isEmpty()) return;
79
80      //display the element to be popped:
81      std::cout<<"Popping: "<<stack[top]<<std::endl;
82
83      //decrement top
84      top--;
85
86
```

```
87  }
88
89  void Top(){
90      //error checking
91      if(isEmpty()) return;
92
93      //display the top element
94      std::cout<<"The top element is: "<<stack[top]<<std::endl;
95
96  }
97
98  void displayAll(){
99      if(isEmpty()) return;
100
101     std::cout<<"Display All"<<std::endl;
102     for(int i = top; i >= 0; i--){
103         std::cout<<stack[i]<<std::endl;
104
105     }
106     return;
107 }
```

Output

```
Enter the number of maximum elements for the new stack (max: 100): 10
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
1
What number you want to add to the stack?: 1
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
1
What number you want to add to the stack?: 2
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
1
What number you want to add to the stack?: 3
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
1
```

```
What number you want to add to the stack?: 4
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
1
What number you want to add to the stack?: 5
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
2
Popping: 5
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
3
The top element is: 4
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
```

```
4
0
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
5
0
Stack Operations:
1. PUSH
2. POP
3. TOP
4. isEmpty
5. isFull
6. Display All
6
Display All
4
3
2
1
```

Code Analysis

For the ILO B1 I built this program to implement a stack using a plain **int** array instead of **std::stack**, giving me manual control over the underlying data with a fixed-size array called **stack** sized by **maxCap**, which I set to **100**. I use **top** as my index tracker, starting it at **-1** to represent an empty stack, and I track two other global variables, **newData** for values I push in and **elements** for the maximum size I let the user choose at runtime. I prototype all my functions, **isEmpty()**, **push()**, **pop()**, **Top()**, **isFull()**, and **displayAll()**, before defining them, and I drive the whole program with a menu loop inside **main()** that reads the user's choice through **std::cin** and dispatches it with a **switch** statement. My **isEmpty()** function checks if **top == -1** and prints "Stack Underflow" while returning **true** if so, and my **isFull()** function checks if **top >= elements-1** and prints "Stack Overflow" while returning **true** if so, both of which I reuse across my other functions as guards. My **push()** function calls **isFull()** first and returns early if it's full, otherwise I prompt for **newData** and store it at **stack[++top]**, correctly incrementing **top** before inserting.

My **pop()** function calls **isEmpty()** first and returns early if empty, otherwise I print the value at **stack[top]** and decrement **top**, correctly discarding the top element. My **Top()** function calls **isEmpty()** first and then prints **stack[top]** without modifying **top**, correctly just peeking. My **displayAll()** function calls **isEmpty()** first and then loops from **top** down to **0**, printing each element, correctly showing the stack from top to bottom. My **switch** statement in **main()** dispatches each menu choice to its matching function, calling **push()** for **1**, **pop()** for **2**, **Top()** for **3**, printing the result of **isEmpty()** for **4**, printing the result of **isFull()** for **5**, and calling **displayAll()** for **6**. Overall, my array-based stack implementation covers **push**, **pop**, **Top**, **isEmpty**, **isFull**, and **displayAll** using straightforward index manipulation on a fixed-size array.

ILO B2
Main.py

listStack.h > push<T>(T)

```cpp
#ifndef LIST_STACK_H
#define LIST_STACK_H


//class node with dat and next members
template <typename T>
class Node {
    public:
    T data;
    Node<T>*next = nullptr;
};

//create a pointer/reference tip that always point to the current top node
template <typename T>
Node<T>* head = nullptr;


//pushing operation:
template <typename T>
void push(T newData){
    //create a new node
    Node<T>* newNode = new Node<T>;
    //assign the data to the newnode
    newNode->data = newData;
    //point the newNodeto the current top
    newNode->next = head<T>;
    //assign the new Top the new Node
    head<T> = newNode;
}
//isEmpty() Operation:
template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

```

```cpp
template <typename T>
T pop(){
    //if the stack is empty error catch:
    if(isEmpty<T>()){
        std::cout<<"Stack underflow"<<std::endl;
    }
    //saving a current top node to a temporary variable
    Node<T>* temp = head<T>;
    //store the data to a temp variable
    T tempval = temp->data;
    //move the head to next node
    head<T> = head<T>->next;
    //free up the memory
    delete temp;
    //return the temporary variable data
    return tempval;
}

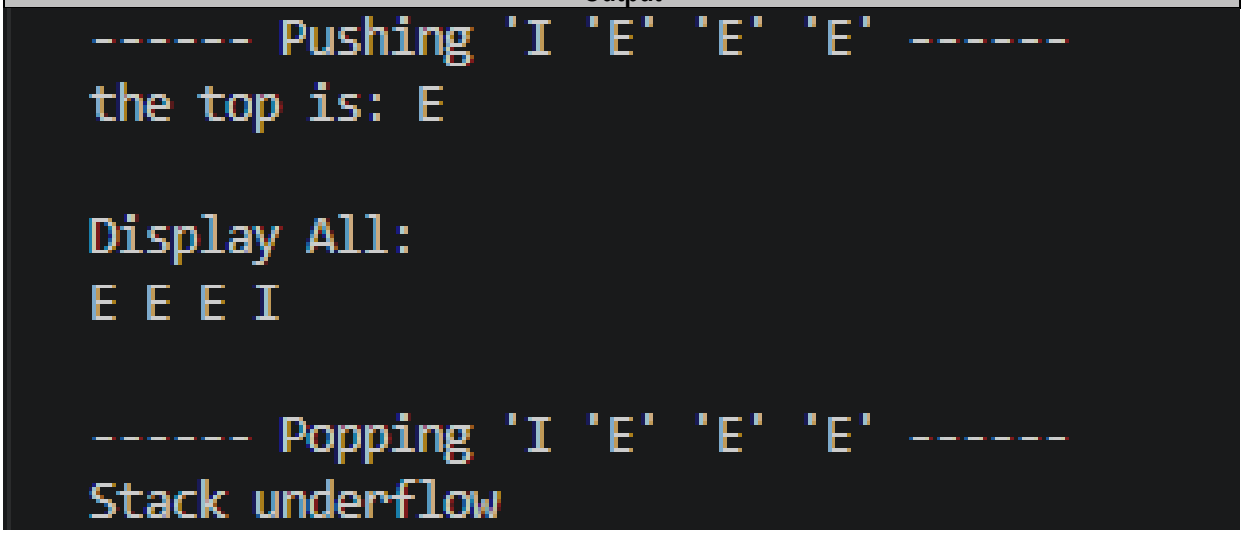
template <typename T>
void Top(){
    //what if the stack is empty
    if(isEmpty<T>()) return;

    std::cout<<"the top is: "<< head<T>->data<<std::endl;
}

template <typename T>
void displayAll(){
    std::cout<<"\nDisplay All: "<<std::endl;

    while(head<T> != nullptr){
        std::cout<<head<T>->data << " ";
        head<T> = head<T>->next;
        }
    std::cout<<"\n"<<std::endl;
    }
#endif
```

Output



Code Analysis

Lastly for the ILO B2 I built this program to implement a generic stack using a **template**-based linked list, where **T** lets me store any data type instead of just **int**. My **Node** class holds two members, **data** to store the value and **next** as a pointer to the next node, and I use **head** as a global template variable that always points to my current top node. My **push()** function takes **newData** as a parameter, creates a new node with **new Node<T>**, assigns **newData** to **newNode->data**, points **newNode->next** to the current **head<T>**, and then reassigns **head<T>** to **newNode**, correctly making the new node the top. My **isEmpty()** function checks whether **head<T> == nullptr** and returns that boolean directly, giving me a quick way to guard my other functions. My **pop()** function checks **isEmpty<T>()** first and prints "Stack underflow" if true, then saves the current **head<T>** into a temporary pointer called **temp**, saves **temp->data** into **tempval**, moves **head<T>** to **head<T>->next**, deletes **temp** to free the memory, and returns **tempval**, correctly removing and returning the top value. My **Top()** function checks **isEmpty<T>()** first and returns early if true, otherwise it prints **head<T>->data**, correctly peeking without modifying the stack. My **displayAll()** function prints a header, then loops through the list while **head<T> != nullptr**, printing each **head<T>->data** and advancing **head<T>** to **head<T>->next** on every iteration, until it reaches the end and prints a trailing newline.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C1 (Using Arrays)

Main.cpp

```cpp
#include <iostream>
#include <string>

const int maxCap = 100;
char stack[maxCap];
int top = -1;

bool isEmpty() {
    return top == -1;
}

void push(char symbol) {
    top++;
    stack[top] = symbol;
}

char pop() {
    char symbol = stack[top];
    top--;
    return symbol;
}

bool MatchingDelimiters(char opening, char closing) {
    if (opening == '(' && closing == ')') return true;
    if (opening == '{' && closing == '}') return true;
    if (opening == '[' && closing == ']') return true;
    return false;
}

```

```
30  int main() {
31      std::string expr;
32      std::cout << "Enter Expression: ";
33      std::cin >> expr;
34
35      bool balanced = true;
36
37      for (int i = 0; i < expr.length(); i++) {
38          char ch = expr[i];
39
40          if (ch == '(' || ch == '{' || ch == '[') {
41              push(ch);
42          }
43          else if (ch == ')' || ch == '}' || ch == ']') {
44              if (isEmpty()) {
45                  balanced = false;
46                  break;
47              }
48              char opening = pop();
49              if (!MatchingDelimiters(opening, ch)) {
50                  balanced = false;
51                  break;
52              }
53          }
54      }
55
56      if (!isEmpty()) {
57          balanced = false;
58      }
59
```

```
60      if (balanced) {
61          std::cout << "Balanced Expression" << std::endl;
62      } else {
63          std::cout << "Unbalanced Expression" << std::endl;
64      }
65
66      return 0;
67  }
```

Output

```
Enter Expression: (A+B)+(C-D)
Balanced Expression
Enter Expression: ((A+B)+(C-D)
Unbalanced Expression
Enter Expression: ((A+B)+[C-D])
Balanced Expression
Enter Expression: ((A+B]+[C-D]}
Unbalanced Expression
```

Code Analysis

For the ILOC1 using arrays I built this program to check whether an expression's delimiters, parentheses, curly braces, and square brackets, are balanced, using a simple **char**-based array stack. I use a fixed-size array called **stack** sized by **maxCap**, which I set to **100**, and I track my position with **top**, starting it at **-1** to represent an empty stack. My **isEmpty()** function checks whether **top == -1** and returns that boolean directly. My **push()** function increments **top** and stores the given **symbol** at **stack[top]**, correctly adding a new opening delimiter to the top. My **pop()** function reads the value at **stack[top]**, decrements **top**, and returns that value, correctly removing and returning the top delimiter. My **MatchingDelimiters()** function takes an **opening** and a **closing** character and checks all three valid pairs, **(** with **)**, **{** with **}**, and **[** with **]**, returning **true** if they match and **false** otherwise. In **main()**, I read the user's input into **expr** using **std::cin**, then loop through each character with **expr[i]**, and whenever I encounter an opening delimiter I call **push(ch)**, and whenever I encounter a closing delimiter I first check **isEmpty()**, setting **balanced** to **false** and breaking if the stack has nothing to match against, otherwise I call **pop()** to get the corresponding **opening** character and pass both to **MatchingDelimiters()**, setting **balanced** to **false** and

breaking if they don't match. After the loop finishes, I also check **isEmpty()** one more time, setting **balanced** to **false** if the stack still has unmatched opening delimiters left over. Finally, I print "Balanced Expression" if **balanced** stays **true** through all these checks, or "Unbalanced Expression" otherwise.

ILO C2 (Using Linked List)

Main.cpp

```cpp
#include <iostream>
#include <string>

struct Node {
    char symbol;
    Node* next;
};

Node* top = nullptr;

bool isEmpty() {
    return top == nullptr;
}

void push(char symbol) {
    Node* newNode = new Node();
    newNode->symbol = symbol;
    newNode->next = top;
    top = newNode;
}

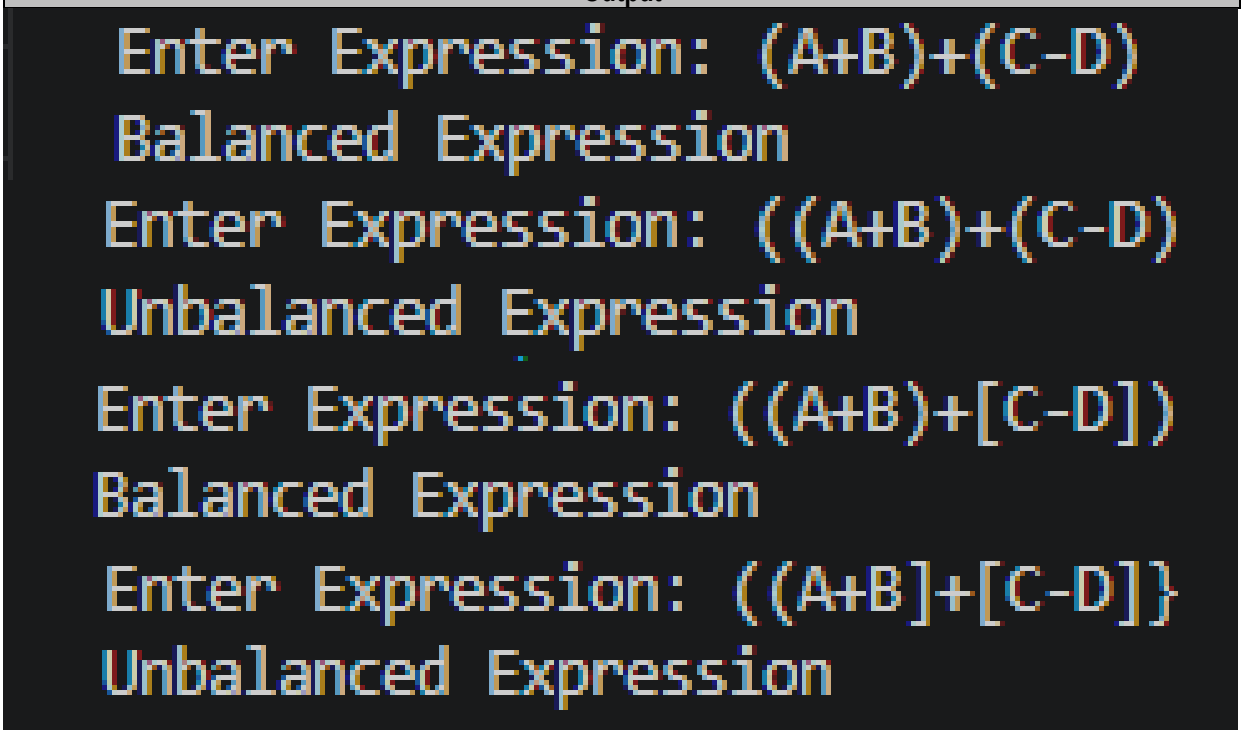
char pop() {
    char symbol = top->symbol;
    Node* temp = top;
    top = top->next;
    delete temp;
    return symbol;
}

```

```
30  bool MatchingDelimiters(char opening, char closing) {
31      if (opening == '(' && closing == ')') return true;
32      if (opening == '{' && closing == '}') return true;
33      if (opening == '[' && closing == ']') return true;
34      return false;
35  }
36
37  int main() {
38      std::string expr;
39      std::cout << "Enter Expression: ";
40      std::cin >> expr;
41
42      bool balanced = true;
43
44      for (int i = 0; i < expr.length(); i++) {
45          char ch = expr[i];
46
47          if (ch == '(' || ch == '{' || ch == '[') {
48              push(ch);
49          }
50          else if (ch == ')' || ch == '}' || ch == ']') {
51              if (isEmpty()) {
52                  balanced = false;
53                  break;
54              }
55              char opening = pop();
56              if (!MatchingDelimiters(opening, ch)) {
57                  balanced = false;
58                  break;
59              }
60          }
61      }
62
```

```
63      if (!isEmpty()) {
64          balanced = false;
65      }
66
67      if (balanced) {
68          std::cout << "Balanced Expression" << std::endl;
69      } else {
70          std::cout << "Unbalanced Expression" << std::endl;
71      }
72
73      return 0;
74  }
```

Output



Code Analysis

For the ILOC2 using linked list I built this program to check whether an expression's delimiters, parentheses, curly braces, and square brackets, are balanced, using a linked-list-based **char** stack instead of an array. I define a **Node** struct with two members, **symbol** to hold the character and **next** as a pointer to the next node, and I use **top** as a global pointer that always points to the current top node, starting it at **nullptr** to represent an empty stack. My **isEmpty()** function checks whether **top == nullptr** and returns that boolean directly. My **push()** function creates a new node with **new Node()**, assigns the given **symbol** to **newNode->symbol**, points **newNode->next** to the current **top**, and reassigns **top** to **newNode**, correctly making the new node the top. My **pop()** function reads **top->symbol** into a local variable, saves the current **top** into a temporary pointer called **temp**, moves **top** to **top->next**, deletes **temp** to free the memory, and returns the saved **symbol**, correctly removing and returning the top delimiter.

My **MatchingDelimiters()** function takes an **opening** and a **closing** character and checks all three valid pairs, **(** with **)**, **{** with **}**, and **[** with **]**, returning **true** if they match and **false** otherwise. In **main()**, I read the user's input into **expr** using **std::cin**, then loop through each character with **expr[i]**, and whenever I encounter an opening delimiter I call **push(ch)**, and whenever I encounter a closing delimiter I first check **isEmpty()**, setting **balanced** to **false** and breaking if the stack has nothing to match against, otherwise I call **pop()** to get the corresponding **opening** character and pass both to **MatchingDelimiters()**, setting **balanced** to **false** and breaking if they don't match. After the loop finishes, I also check **isEmpty()** one more time, setting **balanced** to **false** if the stack still has unmatched opening delimiters left over. Finally, I print "Balanced Expression" if **balanced** stays **true** through all these checks, or "Unbalanced Expression" otherwise.

Tools Analysis:

- How do the different internal representations affect the implementation and usage of the stack?
  The three internal representations trade off convenience, control, and flexibility: **std::stack** handles memory automatically for the easiest usage, my array-based version gives me fast direct indexing via **stack[top]** but locks me into a fixed **maxCap**, and my linked-list version trades that speed and simplicity for dynamic growth through **new**/**delete** on each **Node**. Overall, the array is best when I know my max size ahead of time, the linked list is best when size is unpredictable, and **std::stack** is best whenever I don't need to think about the internals at all.

Conclusion

Through this activity, I learned how to implement and apply the stack data structure across multiple approaches, starting with the built-in **std::stack** container adapter and its **push()**, **pop()**, **top()**, **empty()**, **size()**, and **swap()** operations, then moving into manual implementations using a fixed-size array with a **top** index variable, a dynamic linked list with a **Node** struct and pointer manipulation, and a **template**-based version that generalized my **push()**, **pop()**, **isEmpty()**, **Top()**, and **displayAll()** functions to work with any data type **T**, and I applied all of this to a real-world use case checking whether an expression's delimiters are balanced using both my array-based and linked-list-based **char** stacks, which showed me how a stack naturally models nested structure since the most recent unmatched opening delimiter, tracked through **push(ch)**, always needs to be the first one resolved through **pop()** and **MatchingDelimiters()**, and overall I think I did well in this activity, successfully implementing stacks through three different underlying approaches and applying my understanding to a practical problem, though my area for improvement going forward is deepening my understanding of memory management in my linked-list-based implementation and practicing more advanced stack applications like expression evaluation

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly & Website Forums**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction, Code ideas/assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Fix my wordings on analyzing codes and learned on how a certain function works using forums and ai**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.